

Agile, Scrum & Requirements Engineering

SWE 4401 — Software Requirements Specification

Study Notes: 10 Sept (Thursday) Topics

Contents

1	Agile Methodology	2
1.1	The Four Core Values	2
1.2	Key Principles (selected)	2
1.3	Agile vs. Waterfall	3
2	Scrum	3
2.1	Scrum Roles	3
2.2	Scrum Events (Ceremonies)	3
2.3	Scrum Artifacts	4
3	Product Backlog, Sprint, Definition of Done (DoD), Definition of Ready (DoR)	5
4	MoSCoW Prioritization	5
5	User Story	6
5.1	Standard Template	6
5.2	INVEST Criteria for Good User Stories	6
6	Acceptance Criteria	7
7	Story Points & Planning Poker	7
8	Epic	8
9	Sprint Burndown Chart	8
10	Development Models — RAD and Prototyping	9
10.1	Rapid Application Development (RAD)	9
10.2	Prototyping Model	9
11	Non-Functional Requirements (NFRs)	10
12	Integrated Scenarios & Practice Questions	12
12.1	Scenario 1: CampusEats — A Food Ordering App for IUT	12
12.2	Scenario 2: NirvanaHealth — Decipher the Roles	15
12.3	Scenario 3: BracVerify — A RAD Case Study	18

1 Agile Methodology

Definition

Agile is an iterative, incremental approach to software development that emphasizes flexibility, customer collaboration, and delivering working software in short cycles rather than following a single rigid, front-loaded plan.

Agile arose as a reaction to heavyweight, document-driven processes (like the Waterfall model), which struggled when requirements changed mid-project. It is codified by the **Agile Manifesto (2001)**, which has 4 core values and 12 supporting principles.

1.1 The Four Core Values

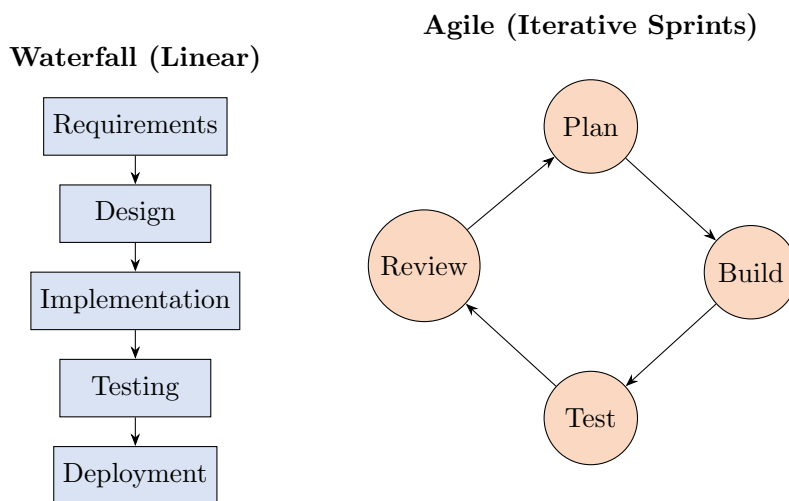
1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a plan

Note: the items on the right still have value — Agile just values the items on the left more.

1.2 Key Principles (selected)

- Highest priority is customer satisfaction through early and continuous delivery.
- Welcome changing requirements, even late in development.
- Deliver working software frequently (weeks rather than months).
- Business people and developers must work together daily.
- Build projects around motivated individuals; give them the environment and trust to get the job done.
- Working software is the primary measure of progress.
- Continuous attention to technical excellence and good design.
- Simplicity — maximizing the amount of work *not* done — is essential.
- Regular reflection (retrospectives) to tune and adjust behavior.

1.3 Agile vs. Waterfall



Example

A university wants a new mobile attendance app. Under **Waterfall**, all requirements are fixed upfront, and the client only sees the app after 6 months. Under **Agile**, the team delivers a working "login + mark attendance" feature in the first 2-week sprint, gets feedback, and adjusts the next sprint's plan — reducing the risk of building the wrong thing.

2 Scrum

Definition

Scrum is the most widely used **Agile framework**. It organizes work into fixed-length iterations called **Sprints** (usually 1–4 weeks), during which a cross-functional team builds a potentially shippable product increment.

2.1 Scrum Roles

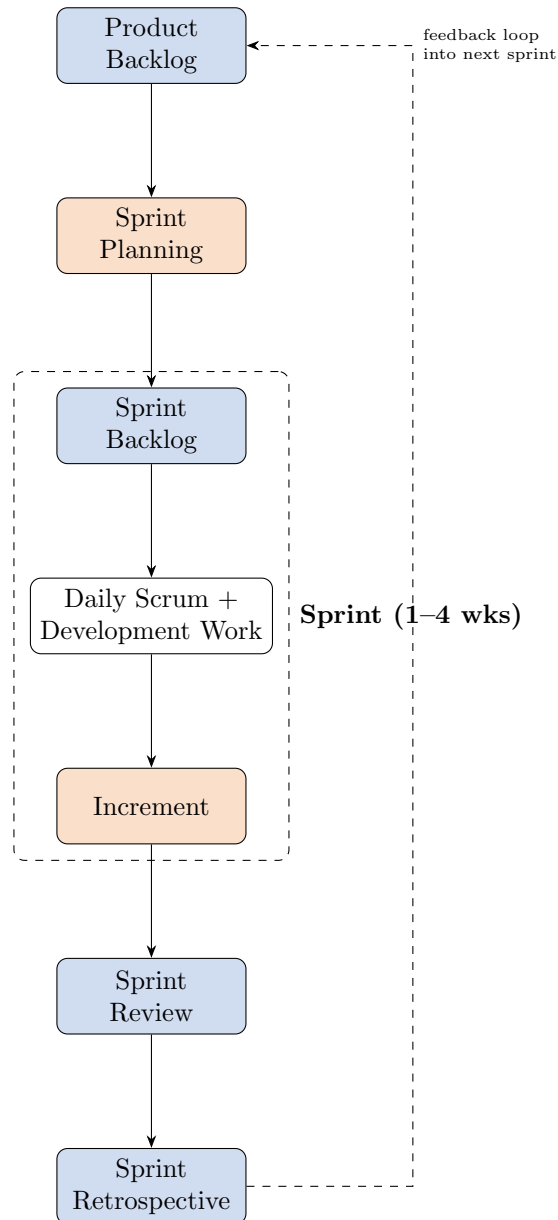
- **Product Owner (PO)** — owns the Product Backlog, defines priorities, represents the customer/business.
- **Scrum Master** — facilitates the process, removes impediments, coaches the team on Scrum practices (not a project manager who assigns tasks).
- **Development Team** — self-organizing, cross-functional group that builds the increment.

2.2 Scrum Events (Ceremonies)

- **Sprint Planning** — team selects backlog items for the sprint and defines a Sprint Goal.
- **Daily Scrum (Stand-up)** — 15-minute daily sync: what I did, what I'll do, blockers.
- **Sprint Review** — demo the increment to stakeholders, gather feedback.
- **Sprint Retrospective** — team reflects on *how* they worked and identifies process improvements.

2.3 Scrum Artifacts

- **Product Backlog** — ordered list of everything that might be needed in the product.
- **Sprint Backlog** — subset of backlog items selected for the current sprint, plus a plan to deliver them.
- **Increment** — the sum of all completed backlog items, meeting the Definition of Done.



3 Product Backlog, Sprint, Definition of Done (DoD), Definition of Ready (DoR)

Product Backlog

A single, dynamic, **prioritized** list of features, bug fixes, technical work, and knowledge acquisition needed to improve the product. It is never "complete" — it evolves as the product and market evolve. Owned and ordered by the Product Owner.

Sprint

A time-boxed iteration (commonly 2 weeks) during which a "Done", usable, and potentially releasable increment is created. Sprints have a consistent duration throughout a project and start immediately after the previous one ends.

Definition of Done (DoD)

A shared, team-wide checklist of criteria that a backlog item must satisfy to be considered **complete**. It creates a common understanding of quality.

Example DoD Checklist

- Code written and peer-reviewed
- Unit tests written and passing ($\geq 80\%$ coverage)
- Integrated into the main branch, builds successfully
- Acceptance criteria verified by QA
- Documentation updated

Definition of Ready (DoR)

A checklist confirming a backlog item is well-understood enough to be pulled into a sprint — i.e. it is **ready to be started**, not ready to be shipped.

Example DoR Checklist

- User story follows INVEST criteria (see Section 5)
- Acceptance criteria are written and agreed upon
- Story is estimated (story points assigned)
- Dependencies identified and resolved/known
- Mockups/designs attached if UI is involved

Key distinction: DoR gates entry *into* a sprint; DoD gates exit *out of* a sprint.

4 MoSCoW Prioritization

Definition

MoSCoW is a prioritization technique used to reach a shared understanding with stakeholders on the importance of requirements. The name is an acronym (extra 'o's added for pronounceability):

Category	Meaning
M — Must have	Non-negotiable; without it, the delivery is a failure.
S — Should have	Important but not vital; painful to leave out but a workaround exists.
C — Could have	Desirable but not necessary; nice-to-have if time/budget allows.
W — Won't have (this time)	Explicitly agreed to be out of scope for this delivery/timebox.

Example — E-commerce Checkout Feature

- **Must:** Users can pay with a credit card.
- **Should:** Users can save a shipping address for next time.
- **Could:** Users can apply a gift-wrap option.
- **Won't (this release):** Users can pay via cryptocurrency.

5 User Story

Definition

A user story is a short, simple description of a feature written from the perspective of the end user. It captures the *who*, *what*, and *why* without prescribing implementation details.

5.1 Standard Template

As a [type of user], I want [some goal] so that [some reason/benefit].

Example

"As a **registered student**, I want to **view my exam routine on the mobile app** so that **I don't miss any exams**."

5.2 INVEST Criteria for Good User Stories

Letter	Meaning
I — Independent	Can be developed/delivered without depending on other stories.
N — Negotiable	Details can be discussed; not a rigid contract.
V — Valuable	Delivers clear value to a user or customer.
E — Estimable	Team can reasonably estimate the effort.
S — Small	Small enough to fit in a single sprint.
T — Testable	Has clear criteria to verify completion.

6 Acceptance Criteria

Definition

Acceptance criteria are the specific, testable conditions a user story must satisfy to be considered "Done" from the customer's point of view. They remove ambiguity and define the boundary of a story.

The most common format is **Gherkin syntax**: Given/When/Then.

Given [*initial context*], **When** [*an action occurs*], **Then** [*expected outcome*].

Example — for the exam-routine story above

- **Given** I am a logged-in student, **When** I open the "Exams" tab, **Then** I see a list of my upcoming exams sorted by date.
- **Given** I have no upcoming exams, **When** I open the "Exams" tab, **Then** I see a message "No upcoming exams."
- **Given** I am not logged in, **When** I try to open the "Exams" tab, **Then** I am redirected to the login page.

7 Story Points & Planning Poker

Story Points

A relative, unit-less measure of the **effort, complexity, and uncertainty** involved in implementing a backlog item — *not* a direct measure of time. Teams typically use a modified Fibonacci sequence (1, 2, 3, 5, 8, 13, 21, ...) because the growing gaps force meaningful distinctions between estimates as size increases.

Planning Poker

A consensus-based, gamified estimation technique. Each team member privately selects a card representing their story-point estimate; all cards are revealed simultaneously to avoid anchoring bias. If estimates diverge significantly, the highest and lowest estimators explain their reasoning, and the team re-votes until convergence.



Planning Poker deck (Fibonacci-like values)

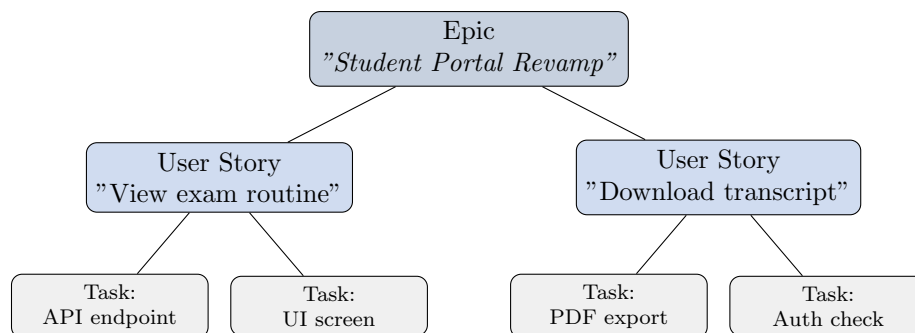
Example

The team estimates "Add password reset via email." Alice votes 3, Bob votes 3, Carol votes 8 (she recalls SMTP integration issues from a past project). The team discusses Carol's concern, agrees there's real risk, and converges on 5.

8 Epic

Definition

An epic is a large body of work that is too big to fit in a single sprint. It is broken down into smaller, manageable **user stories**, which may themselves be broken into **tasks** during sprint planning.



Example

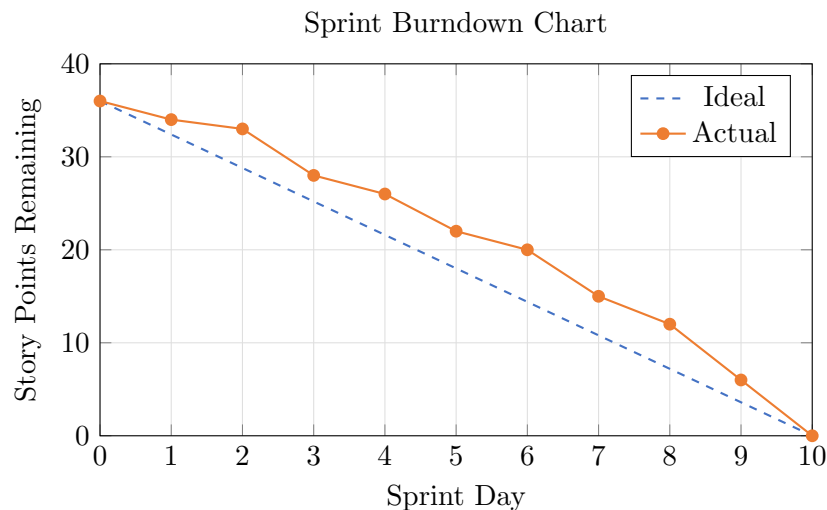
Epic: "Student Portal Revamp" → broken into stories like "View exam routine," "Download transcript," "Update contact info" → each story broken into tasks like "design API," "build UI," "write tests."

9 Sprint Burndown Chart

Definition

A sprint burndown chart is a visual tool that tracks the amount of **work remaining** (usually in story points or hours) against **time** throughout a sprint. It helps the team see at a glance whether they are on track to finish everything by the sprint's end.

- **X-axis:** days of the sprint.
- **Y-axis:** remaining work (story points/hours).
- **Ideal line:** a straight diagonal line assuming steady, uniform progress.
- **Actual line:** the real remaining work, plotted daily; it is rarely a perfectly straight line.



Reading the chart

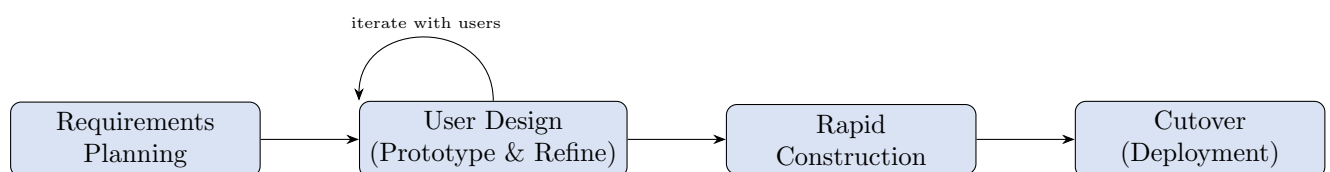
If the **actual** line sits *above* the ideal line, the team is behind schedule (a scope-cut or impediment-removal conversation may be needed). If it dips *below*, the team is ahead of pace. A flat stretch usually signals a blocker (e.g., waiting on a third-party API) rather than genuine idleness.

10 Development Models — RAD and Prototyping

10.1 Rapid Application Development (RAD)

Definition

RAD is a development model that prioritizes rapid prototyping and quick feedback over long, detailed planning phases. It typically compresses timelines using reusable components, code generators, and heavy user involvement, aiming to deliver in weeks/months rather than years.



Best suited for: projects with well-understood business requirements, modest technical risk, and a need for speed (e.g., internal admin dashboards, small business tools). **Not ideal for:** highly complex, novel systems, or projects that cannot be modularized.

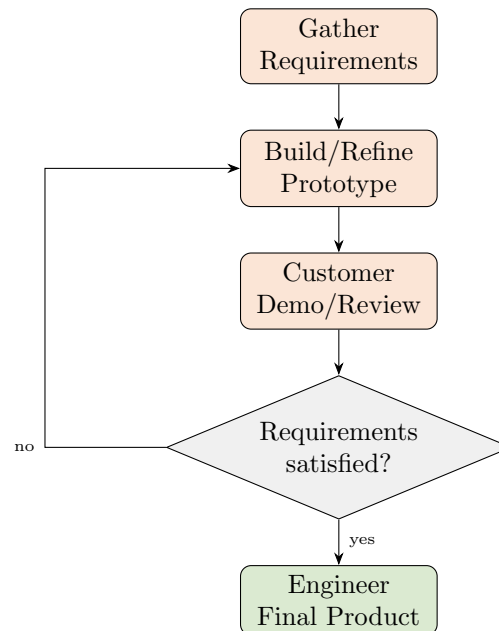
10.2 Prototyping Model

Definition

The prototyping model builds a scaled-down, working (or partially working) mock-up of the system early on so that users can interact with it, clarify requirements, and reduce misunderstanding before full-scale development begins.

Types of prototypes:

- **Throwaway (Rapid) Prototyping** — built quickly to validate requirements, then discarded.
- **Evolutionary Prototyping** — iteratively refined until it becomes the final system.

**Example**

A bank wants a new loan-approval interface but isn't sure what fields loan officers actually need. A **throwaway prototype** (clickable Figma mockup) is shown to officers, refined over three rounds of feedback, then discarded once the real requirements are locked in and full development starts from scratch.

11 Non-Functional Requirements (NFRs)

Definition

Non-functional requirements describe **how** a system should behave rather than **what** it should do. They constrain the solution (quality attributes) rather than specifying a particular function, and are just as critical to project success as functional requirements.

Category	Example
Performance	The system must respond to search queries within 2 seconds under 500 concurrent users.
Security	All passwords must be stored using bcrypt hashing with a minimum cost factor of 12.
Usability	A new user must be able to complete registration in under 3 minutes without help.
Reliability/Availability	The system must maintain 99.9% uptime, excluding scheduled maintenance.
Scalability	The system must support a 10× increase in users without architectural redesign.
Maintainability	Code must follow the team's style guide and pass static analysis with zero critical issues.
Portability	The mobile app must run on both Android 10+ and iOS 14+.
Compliance/Legal	Student data must be handled per the institution's data-protection policy.

Functional vs. Non-Functional (contrast)

Functional: "Users can reset their password via email."

Non-functional: "Password-reset emails must be delivered within 30 seconds and the reset link must expire after 15 minutes."

12 Integrated Scenarios & Practice Questions

12.1 Scenario 1: CampusEats — A Food Ordering App for IUT

Scenario

The Computer Club at IUT is building **CampusEats**, a mobile app that lets students pre-order food from campus cafeterias to skip queues between classes. The team has 5 developers, a Product Owner (club president), and a Scrum Master, and works in **2-week sprints**.

At the initial planning meeting, the Product Owner proposes the following **epic**: *"Enable students to browse, order, and pay for food from any campus cafeteria."* The team breaks this into stories and begins backlog refinement using **MoSCoW**, discusses **non-functional requirements** demanded by the cafeteria staff (who worry about order-spikes during lunch rush), and starts the first sprint with a **prototyping** exercise for the ordering screen because nobody is sure how students will want to filter menu items.

By the end of Sprint 1, the team held daily stand-ups, tracked a **burndown chart**, and used **planning poker** to estimate stories in **story points**. Not everything got finished — the "Payment Gateway Integration" story is only half-built.

Q1 — Agile Methodology

Why might the Computer Club prefer Agile over a Waterfall plan for CampusEats, given that they don't yet know exactly how students will want to browse the menu?

Sample Answer: Since key UX details (how students filter/browse food) are unknown upfront, a Waterfall plan would lock in a possibly-wrong design for months. Agile allows the team to build a thin slice (e.g., a browsing prototype), get real student feedback within one sprint, and adjust — reducing the risk of investing in the wrong interface.

Q2 — Epic to User Stories

Break the epic "Enable students to browse, order, and pay for food from any campus cafeteria" into at least three well-formed user stories.

Sample Answer:

- As a student, I want to browse a cafeteria's menu by category so that I can quickly find what I want to eat.
- As a student, I want to add items to a cart and place an order so that I can skip the physical queue.
- As a student, I want to pay for my order using my student ID balance so that I don't need cash.
- As a cafeteria staff member, I want to see incoming orders on a kitchen display so that I can prepare them in sequence.

Q3 — Acceptance Criteria

Write Gherkin-style acceptance criteria for the story "As a student, I want to add items to a cart and place an order so that I can skip the physical queue."

Sample Answer:

- Given I have selected at least one menu item, When I tap "Add to Cart," Then the item appears in my cart with correct price and quantity.
- Given my cart has items, When I tap "Place Order," Then I receive an order confirmation with an estimated pickup time.
- Given the cafeteria has marked an item "out of stock," When I try to add it to the cart, Then I see an error message and cannot add it.

Q4 — MoSCoW Prioritization

Classify these four features for CampusEats' first release using MoSCoW: (a) card/mobile-banking payment, (b) push notification when order is ready, (c) student ID balance payment, (d) loyalty points/rewards program.

Sample Answer: Must: (c) Student ID balance payment — this is the core, expected payment path for the campus. **Should:** (b) Push notification when order is ready — big UX improvement, but students could just check the app. **Could:** (a) Card/mobile-banking payment — nice alternative but ID balance already covers the core need. **Won't (this release):** (d) Loyalty rewards program — adds complexity with no impact on the core "skip the queue" value proposition.

Q5 — Definition of Ready vs. Definition of Done

The "Payment Gateway Integration" story was pulled into Sprint 1 but ended up only half-finished. What DoR check might the team have skipped, and what does the DoD tell them about the story's current state?

Sample Answer: The team likely skipped confirming that **dependencies were resolved** in the DoR (e.g., they may not have had test credentials for the payment gateway sandbox, or the API documentation wasn't reviewed before estimation) — meaning the story wasn't actually "Ready." According to the DoD, since the code isn't merged/tested/reviewed, the story is **not Done** and must be carried over (not counted as completed) into the next sprint; the team should re-estimate remaining work rather than mark partial credit.

Q6 — Story Points & Planning Poker

During planning poker for "Payment Gateway Integration," one developer votes 3, another votes 13. What should the team do, and why does this large gap matter?

Sample Answer: A large spread signals a difference in *understanding*, not just opinion — the low estimator may not realize the payment gateway requires PCI-compliance handling or sandbox setup, while the high estimator may be aware of past integration pain. The two should explain their reasoning to the group, and the team re-votes after clarifying scope/unknowns, converging on a shared, better-informed estimate (e.g., settling on 8).

Q7 — Sprint Burndown Chart

If the "Payment Gateway Integration" story stalls for 3 straight days, what would you expect to see on the burndown chart, and what should the Scrum Master do?

Sample Answer: The **actual** line would flatten (no story points burned down) for those 3 days while the **ideal** line keeps descending, so the actual line drifts increasingly above the ideal line. The Scrum Master should treat this as a signal of an impediment (e.g., missing sandbox credentials) and proactively investigate/remove the blocker in the Daily Scrum rather than waiting for the Sprint Review.

Q8 — Development Model Choice

Why did the team choose a throwaway prototype for the "browse menu" screen instead of jumping directly into building it inside a normal sprint?

Sample Answer: Because the team was genuinely **uncertain how students want to filter/browse the menu**, building the real, production-quality screen first risks wasted engineering effort if the design is wrong. A quick, disposable prototype (e.g., a clickable mockup) lets them validate the UX cheaply with real students, and once the correct design is confirmed, they build the polished version from scratch (or evolve the prototype, if they'd chosen evolutionary prototyping instead).

Q9 — Non-Functional Requirements

The cafeteria staff are worried about order-spikes during lunch rush (12:00–1:00 PM). Write one measurable NFR that addresses this concern.

Sample Answer: "The system must successfully process at least 300 concurrent order submissions during peak lunch hours (12:00–1:00 PM) with a 95th-percentile response time under 3 seconds, and must not drop or duplicate any order." This is a **performance/reliability** NFR — it doesn't describe a new feature, but constrains how the ordering feature must behave under load.

12.2 Scenario 2: NirvanaHealth — Decipher the Roles

This scenario deliberately never uses the words "Product Owner," "Scrum Master," or "Development Team." Read carefully — you are expected to work out who is playing which role from what they do, not from a label.

Scenario

NirvanaHealth, a startup incubated at IUT's entrepreneurship cell, has partnered with three local clinics to launch a telemedicine platform before the next monsoon season. **Farah Ahmed**, who has spent the last decade managing operations across all three clinic branches, is the only person authorized to decide what actually ships to patients and doctors — she meets with the clinic directors every Friday to translate their complaints and requests into a single, ranked wishlist that she keeps updating herself. She has one big ask, jotted at the top of her notebook: *"Let patients consult a doctor and get a prescription without ever visiting the clinic."* Everything else in her notebook is written underneath that line, broken into smaller pieces.

Six developers work out of a cramped office on the third floor. Every morning at 9:15, they gather around a whiteboard for twelve minutes, each person answering three questions before returning to their desks. A seventh person, **Rafiq**, never touches the whiteboard's code items directly — he times the meeting, notices when a developer has been stuck on the same task for three days running, and spends his afternoons on the phone with the IT department begging for faster server access so the team isn't blocked. When two developers clashed over whether to rebuild the appointment calendar from scratch, Rafiq didn't decide for them — he just made sure they talked it through openly in the next retrospective.

At the estimation meeting on Tuesday, the team was shown a card describing: *"A patient can request a same-day video consultation with any available doctor."* Three developers quietly held up cards reading 5, one held up 13, and two held up 8. Rafiq asked the developer holding 13 to explain, and she mentioned that the video infrastructure had never been tested with more than two simultaneous calls. After a short discussion the team re-voted and landed on 8 unanimously.

Farah has told the team that, for the monsoon-season launch, patients absolutely must be able to book and pay for a consultation, and doctors absolutely must be able to see a patient's basic medical history before the call starts. She'd like patients to be able to rate their doctor afterward, but admits that could wait if the team runs out of time. She has flatly ruled out building an in-app pharmacy delivery feature for this release, telling the team to revisit it "maybe next year."

Q10 — Deciphering Scrum Roles

From the passage, identify who is playing the Product Owner, Scrum Master, and Development Team roles at NirvanaHealth. Point to the specific details that reveal each role.

Sample Answer:

- **Farah Ahmed = Product Owner.** She is the sole person authorized to decide what ships, she gathers and translates stakeholder input (clinic directors) into a single ordered wishlist (the Product Backlog), and she sets priorities for the release (Must/Could/Won't statements). This is textbook Product Owner behavior.
- **Rafiq = Scrum Master.** He facilitates and time-boxes the 12-minute morning huddle (the Daily Scrum), watches for team members stuck on a task (imped-

iments) and works to remove blockers (chasing server access), and protects the team's self-organization by *not* making the calendar-rebuild decision himself — instead ensuring it's resolved in the Retrospective. None of this involves owning the backlog or writing code.

- **The six developers = Development Team.** They are the ones physically building the product, answering the three daily-scrum questions, and holding the estimation (Planning Poker) cards.

Q11 — Identify the Epic and Decompose It

What is the epic underlying NirvanaHealth's project? Break it into at least four well-formed user stories.

Sample Answer: Epic: "Let patients consult a doctor and get a prescription without ever visiting the clinic." (It's too large for one sprint and sits at the top of Farah's backlog, with everything else written underneath it as smaller pieces — classic epic behavior.)

- As a patient, I want to book a same-day video consultation with an available doctor so that I don't have to visit the clinic in person.
- As a patient, I want to pay for my consultation online so that I can complete the booking without a physical visit.
- As a doctor, I want to view a patient's basic medical history before the call so that I can make an informed diagnosis remotely.
- As a patient, I want to receive a digital prescription after my consultation so that I can get medicine without an in-person appointment.
- As a patient, I want to rate my doctor after the consultation so that I can share feedback on my experience.

Q12 — MoSCoW Prioritization

Using Farah's statements, classify these four features with MoSCoW: (a) booking & paying for a consultation, (b) doctors viewing medical history before the call, (c) patients rating their doctor, (d) in-app pharmacy delivery.

Sample Answer: Must: (a) booking & payment and (b) medical-history viewing — Farah says both are "absolutely must" for the launch. **Could:** (c) doctor rating — she "would like" it but explicitly says it "could wait." **Won't (this release):** (d) pharmacy delivery — flatly ruled out until "maybe next year." *Note:* the passage gives no clear **Should** item — not every real scenario neatly fills all four MoSCoW buckets, and forcing one in would misrepresent what Farah actually said.

Q13 — Reading a Planning Poker Round

Walk through what happened at Tuesday's estimation meeting. What does it demonstrate about the purpose of Planning Poker?

Sample Answer: Votes came in as three 5s, two 8s, and one outlier 13. Instead of averaging the numbers or having Rafiq (who isn't even a developer) decide, the team asked the *outlier* to explain her reasoning — she flagged a real, concrete risk (untested

video infrastructure beyond two simultaneous calls) that the other estimators hadn't considered. This surfaced hidden complexity *before* the sprint started, and the re-vote (converging on 8) reflects the team's updated, shared understanding of the story's true size — exactly why Planning Poker asks for private, simultaneous reveals rather than open discussion first (it prevents anchoring on the first number spoken).

Q14 — Acceptance Criteria

Write Gherkin-style acceptance criteria for the story "As a patient, I want to book a same-day video consultation with an available doctor so that I don't have to visit the clinic in person," making sure to reflect the same-day/availability constraint.

Sample Answer:

- Given a patient is logged in, When they search for doctors today, Then only doctors marked "available now" are shown in the results.
- Given a patient selects an available doctor, When they confirm the booking, Then a same-day video slot is reserved and a confirmation is sent immediately.
- Given no doctors are available at the requested time, When the patient searches, Then the system displays "No doctors available right now — please check back shortly" instead of an empty list.

Q15 — Scrum Master Judgment Call

When two developers disagreed about rebuilding the appointment calendar, Rafiq refused to make the call himself. Was this correct Scrum Master behavior? Why or why not?

Sample Answer: Yes. The Development Team is meant to be **self-organizing** — technical, "how do we build it" decisions belong to the developers, not the Scrum Master. Rafiq's role is to create the conditions for the team to resolve the disagreement itself (routing it into the Retrospective, a safe forum for exactly this kind of process/technical friction), not to act as a manager who overrules them.

12.3 Scenario 3: BracVerify — A RAD Case Study

Scenario

BracVerify, a fintech compliance vendor, has just signed a rushed contract with a mid-size bank that needs a new "Know Your Customer" (KYC) document-verification portal live in **eight weeks** — a deadline set by an upcoming regulatory audit, not by the engineering team. The bank's compliance officers already know exactly what fields and validation rules they need (they've used a paper checklist for years), so there's little mystery about the underlying business logic.

BracVerify's lead architect proposes skipping the usual multi-month specification-and-sign-off process. Instead, she pulls a KYC form-builder, a document-OCR component, and an authentication module from the company's internal library of pre-built modules — all reused from three previous bank projects — and assembles a rough working version of the portal in **four days**. Every Thursday for the next six weeks, the compliance officers sit with two developers in a "design lab," pointing at the running screen and asking for fields to be renamed, reordered, or removed on the spot, while the developers adjust the interface right in front of them. By week six, the bank's officers are effectively using a version of the portal that looks and behaves almost exactly like what eventually ships.

In week seven, the team hardens security, load-tests the OCR pipeline against 10,000 sample documents, and migrates real customer data. In week eight, the old paper-based process is switched off entirely and the new portal goes live for all branches simultaneously. Meanwhile, a different BracVerify team was asked to build a fraud-detection engine using a novel machine-learning model nobody on the team has implemented before, where even the compliance department isn't sure what "suspicious" should mean until they see real transaction patterns. The lead architect advised strongly *against* using the same rapid, modular approach for that second project.

Q16 — Identify the Model

Which development model is BracVerify using for the KYC portal? Point to at least four specific details in the passage that support your answer.

Sample Answer: RAD (Rapid Application Development). Evidence: (1) an aggressive, externally-fixed 8-week deadline driving the choice of process; (2) heavy reuse of pre-built, reusable components (form-builder, OCR module, auth module) from earlier projects instead of building from scratch; (3) a rough working version assembled in just four days — rapid initial construction; (4) continuous, hands-on user involvement in a weekly "design lab" where the interface is refined live with real users rather than through written sign-off documents; (5) a distinct final "cutover" where the old paper process is retired and the new system goes live for everyone at once.

Q17 — Map the Timeline to RAD's Phases

Map the narrative's timeline onto the four classic RAD phases: Requirements Planning, User Design, Rapid Construction, and Cutover.

Sample Answer:

- **Requirements Planning:** Effectively already done before day one — the compliance officers' long-standing paper checklist means the business rules were already

well understood, so this phase was brief rather than skipped.

- **User Design:** Weeks 1–6, the recurring Thursday "design lab" sessions where officers and developers iteratively refine the interface together.
- **Rapid Construction:** The initial four-day assembly from reusable components, continuing in parallel with User Design as live changes are implemented.
- **Cutover:** Weeks 7–8 — security hardening, load-testing, data migration, and the simultaneous switch-off of the old paper process across all branches.

Q18 — Why RAD Was Rejected for the Second Project

Explain, using the fraud-detection counter-example, why RAD was not recommended for that second project. Refer to RAD's known limitations.

Sample Answer: RAD works best when requirements are already well understood and reusable components/technical risk are low, so a team can assemble and refine quickly. The fraud-detection project fails both conditions: the ML model is novel and unimplemented (no reusable components, high technical risk and uncertainty), and even the business side doesn't yet know what "suspicious" means until real data is observed (requirements aren't just unwritten — they're genuinely unknown). RAD is unsuitable for high-novelty, high-risk, discovery-driven work like this; such a project needs research/experimentation cycles, not rapid assembly.

Q19 — The Role of User Involvement

The compliance officers reshaped the interface "on the spot" during the weekly design lab. Which core RAD characteristic does this illustrate, and why did it matter for the 8-week deadline?

Sample Answer: This illustrates RAD's hallmark of **intensive, continuous end-user involvement**. Because the business logic was already known, going straight to an interactive, adjustable prototype let the team skip slow, indirect requirements documents and lengthy sign-off cycles — real users could correct misunderstandings immediately, which is exactly what a compressed 8-week timeline demands.

Q20 — Contrasting with a Document-First Approach

If BracVerify had instead spent the first three weeks writing a full Software Requirements Specification (SRS) and waiting for the bank's sign-off before writing any code, what risk would this introduce given the 8-week deadline?

Sample Answer: Spending three of eight weeks purely on a written document, with no working software, would leave too little time for construction, hardening, load-testing, and data migration — and any misunderstanding buried in the document (easy to miss on paper, much harder to miss on a live screen) would only surface *after* sign-off, forcing costly rework with almost no schedule left to absorb it. RAD avoided this entirely by merging requirement discovery and construction into the same fast, iterative loop instead of sequencing them.